LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 09 - STATIC CLASS DAN COLLECTION



Jury Maxwell

103132400023

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Konsep static dan collection merupakan fondasi penting dalam pemrograman Java. Fitur static mengelola data dan fungsi pada tingkat kelas. Collection memfasilitasi pengelolaan sekumpulan data dinamis secara fleksibel. Kombinasi keduanya menghasilkan struktur kode yang efisien dan modular.

1. **Static Variable:** Variabel ini merupakan atribut milik kelas yang dibagi bersama seluruh objek. Nilai variabel ini seragam untuk semua instance yang dibuat. Pengaksesannya tidak memerlukan instansiasi objek terlebih dahulu.
2. **Static Method:** Metode ini dapat dipanggil langsung melalui nama kelas tanpa objek. Metode static hanya dapat memanggil fungsi dan variabel static lainnya. Pendekatan ini sangat cocok untuk membuat fungsi utilitas pendukung.
3. **Collection Framework:** Collection menyediakan struktur data dinamis untuk menyimpan banyak objek sekaligus. Struktur ini menggantikan keterbatasan array statis yang berukuran tetap. Contoh kelas implementasinya meliputi ArrayList, LinkedList, dan HashSet.
4. **Generics:** Generics memastikan tipe data objek di dalam koleksi terdefinisi jelas. Fitur ini memberikan jaminan keamanan tipe saat tahap kompilasi. Hal ini mencegah potensi error tipe data saat runtime.
5. **Interface Comparable dan Comparator:** Comparable digunakan untuk menentukan urutan alami bawaan pada kelas entitas. Kelas harus mengimplementasikan metode compareTo untuk membandingkan objek. Comparator digunakan untuk membuat strategi pengurutan kustom secara fleksibel.
6. **Java Stream API:** Stream API memproses elemen koleksi menggunakan pendekatan fungsional dan deklaratif. Fitur ini mempermudah operasi penyaringan data dan pencarian objek. Ekspresi lambda mendukung penulisan kode yang jauh lebih ringkas.

B. Guided

1. Penerapan Static Variable pada Class Share

File: Share.java

```java
package guided.gd1;

public class Share {
    private int privateInt;
    private static int staticInt;

    public Share(int pr, int si){
        privateInt = pr;
        staticInt = si;
    }
```

```
    public String toString(){
        return privateInt + " " + staticInt;
    }
}
```

File: Driver.java

```
package guided.gd1;

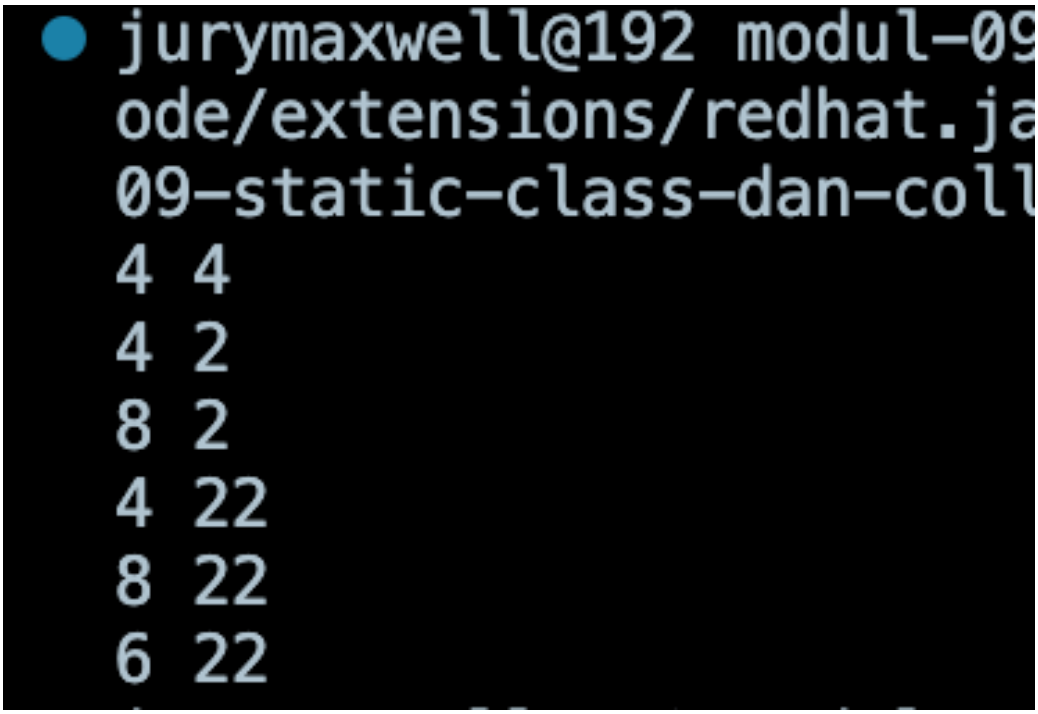
public class Driver {

    public static void main(String[] args) {
        Share s1 = new Share(4, 4);
        System.out.println(s1.toString());

        Share s2 = new Share(8, 2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());

        Share s3 = new Share(6, 22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
}
```

Output :



2. **Penerapan Static Method pada Class CounterMachine**

File: CounterMachine.java

```java
package guided.gd2;

public class CounterMachine {
    static int counter = 0;
    public static void count(){
        counter++;
    }
}
```

File: Driver.java

```java
package guided.gd2;

public class Driver {

    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                CounterMachine.count();
            }
        }
        System.out.println("Counter: " + CounterMachine.counter);
    }
}
```

Output :

```
jurymaxwell@192 modul-09-static-class-dar
ceptionMessages -cp /Users/jurymaxwell/Co
Counter: 5
```

3. **Operasi dan Teknik Iterasi pada Collection List**

Guided 3: Iteration

File: Main.java

```java
package guided.gd3.iteration;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> listBuah = new ArrayList<>();
```

```
        // 2. Menambahkan elemen ke dalam List
        listBuah.add("Apel");
        listBuah.add("Mangga");
        listBuah.add("Jeruk");
        listBuah.add("Pisang");

         // 3. Operasi utilitas List
        System.out.println("Total Elemen: " + listBuah.size());

        System.out.println("Apakah ada 'Mangga'? " +
listBuah.contains("Mangga"));

        // 4. menghapus data
        listBuah.remove("Pisang");

        // --- Teknik 1: For loop tradisional (indeks) ---
        System.out.println("\n--- 1. Normal For-Loop (Indeks) ---");
            for (int i = 0; i < listBuah.size(); i++) {
                System.out.println("Index " + i + ": " +
listBuah.get(i));
        }

        // --- Teknik 2: Enhanced for-each loop ---
        System.out.println("\n--- 2. Enhanced For-Loop (For-Each) ---");
        for (String buah : listBuah) {
            System.out.println(buah);
        }

        // --- Teknik 3: Iterator Interface ---
        System.out.println("\n--- 3. Menggunakan Iterator ---");
        Iterator<String> itr = listBuah.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }

        // --- Teknik 4: Lambda Expression (Java 8+) ---
        System.out.println("\n--- 4. Lambda Expression ---");
        listBuah.forEach(buah -> System.out.println(buah));

        // --- Teknik 5: Method Reference (Java 8+) ---
        System.out.println("\n--- 5. Method Reference ---");
        listBuah.forEach(System.out::println);
    }

}
```

Output :

```
jurymaxwell@192 modul-09-static-class-da
ode/extensions/redhat.java-1.58.0-darwin
09-static-class-dan-collection/project/t
Total Elemen: 4
Apakah ada 'Mangga'? true

--- 1. Normal For-Loop (Indeks) ---
Index 0: Apel
Index 1: Mangga
Index 2: Jeruk

--- 2. Enhanced For-Loop (For-Each) ---
Apel
Mangga
Jeruk

--- 3. Menggunakan Iterator ---
Apel
Mangga
Jeruk

--- 4. Lambda Expression ---
Apel
Mangga
Jeruk

--- 5. Method Reference ---
Apel
Mangga
Jeruk
```

Guided 3: Sorting

File: Employee.java

```java
package guided.gd3.sorting;

public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;

    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
```

```
    public String getName() {
        return name;
    }

    public int getSalary() {
        return salary;
    }

    // Default sorting: Berdasarkan nama (A-Z)
    @Override
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return String.format("Employee[name=%-6s, salary=%d]", name,
salary);
    }
}
```

File: SalaryComparator.java

```
package guided.gd3.sorting;

import java.util.Comparator;

public class SalaryComparator implements java.util.Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return Integer.compare(e1.getSalary(), e2.getSalary());
    }

}
```

File: MainSort.java

```
package guided.gd3.sorting;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class MainSort {
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList<>();
        listEmp.add(new Employee("Bobby", 3000));
        listEmp.add(new Employee("Erick", 1600));
```

```
        listEmp.add(new Employee("Rey", 2500));
        listEmp.add(new Employee("Anna", 3500));

        System.out.println("--- Data Awal ---");
        listEmp.forEach(System.out::println);

        // Sorting 1: Natural Order via Comparable (compareTo)
        Collections.sort(listEmp);
        System.out.println("\n--- Sorted by Name (Comparable / Natural
Order) ---");
        listEmp.forEach(System.out::println);

        // Sorting 2: Custom Order via Comparator (compare)
        Collections.sort(listEmp, new SalaryComparator());
        System.out.println("\n--- Sorted by Salary (Comparator / Custom
Order) ---");
        listEmp.forEach(System.out::println);
    }
}
```

Output :

```
jurymaxwell@192 modul-09-static-class-dan-collection
ceptionMessages -cp /Users/jurymaxwell/College/Pemrog
--- Data Awal ---
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Anna  , salary=3500]

--- Sorted by Name (Comparable / Natural Order) ---
Employee[name=Anna  , salary=3500]
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]

--- Sorted by Salary (Comparator / Custom Order) ---
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Bobby , salary=3000]
Employee[name=Anna  , salary=3500]
```

Guided 3: Filtering

File: Mahasiswa.java

```java
package guided.gd3.filtering;

public class Mahasiswa {
    private String nim;
    private String nama;
    private double ipk;

    public Mahasiswa(String nim, String nama, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.ipk = ipk;
    }

    public String getNim() {
        return nim;
    }

    public String getNama() {
        return nama;
    }

    public double getIpk() {
        return ipk;
    }

    @Override
    public String toString() {
        return String.format("Mhs[NIM=%s, Nama=%-6s, IPK=%.2f]", nim,
nama, ipk);
    }
}
```

File: MainFilter.java

```java
package guided.gd3.filtering;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class MainFilter {
    public static void main(String[] args) {
        List<Mahasiswa> list = new ArrayList<>();
        list.add(new Mahasiswa("001", "Bobby", 3.25));
        list.add(new Mahasiswa("002", "Erick", 2.80));
```

```
        list.add(new Mahasiswa("003", "Rey", 3.75));
        list.add(new Mahasiswa("004", "Anna", 3.90));

        System.out.println("=== 1. FILTERING (IPK >= 3.50) ===");

        // --- Pendekatan 1: Imperatif (Konvensional) ---
        System.out.println("Cara Konvensional:");
        List<Mahasiswa> mhsCumlaudeKonven = new ArrayList<>();
        for (Mahasiswa m : list) {
            if (m.getIpk() >= 3.50) {
                mhsCumlaudeKonven.add(m);
            }
        }
        for (Mahasiswa m : mhsCumlaudeKonven) {
            System.out.println(m);
        }

        // --- Pendekatan 2: Deklaratif (Stream API) ---
        System.out.println("\nCara Stream API:");
        List<Mahasiswa> mhsCumlaudeStream = list.stream()
                .filter(m -> m.getIpk() >= 3.50)
                .collect(Collectors.toList());
        mhsCumlaudeStream.forEach(System.out::println);

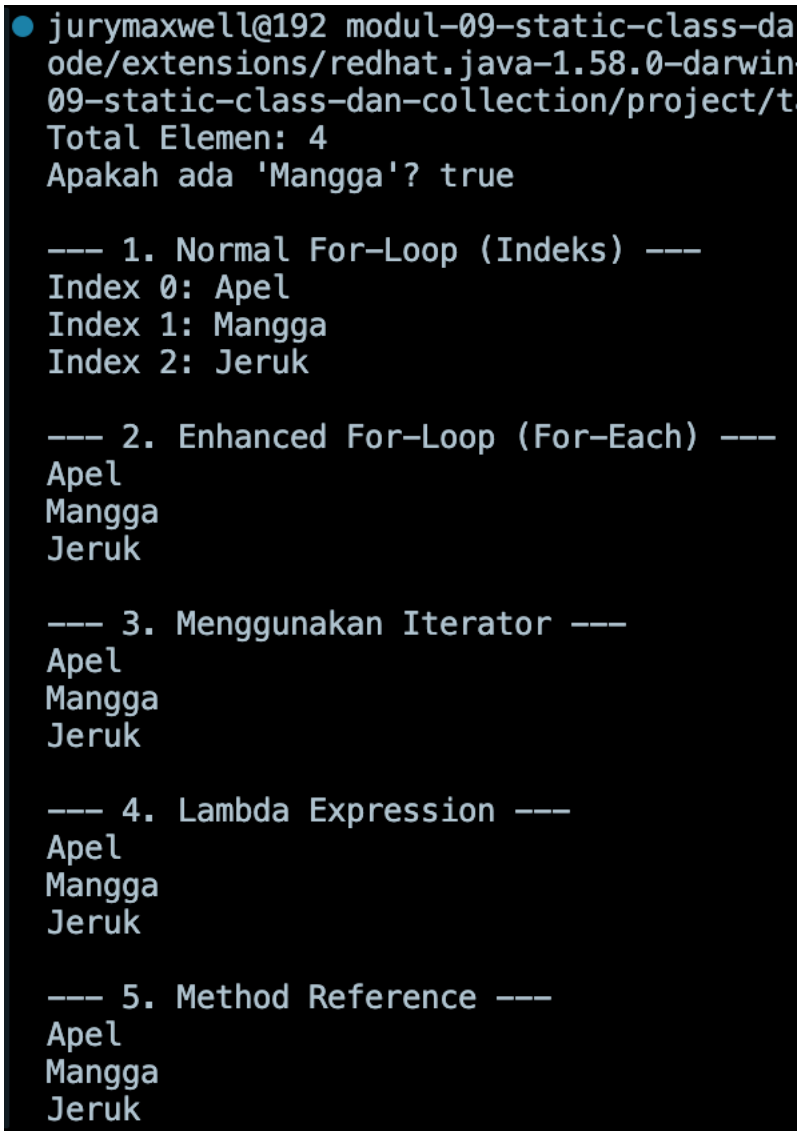
        System.out.println("\n=== 2. SEARCHING / PENCARIAN TUNGGAL ===");

        // --- Pencarian Konvensional ---
        System.out.println("Cara Konvensional (Cari 'Erick'):");
        Mahasiswa targetKonven = null;
        for (Mahasiswa m : list) {
            if (m.getNama().equalsIgnoreCase("Erick")) {
                targetKonven = m;
                break; // Hentikan loop saat ditemukan
            }
        }
        System.out.println(targetKonven != null ? targetKonven : "Data
tidak ditemukan");

        // --- Pencarian via Stream API ---
        System.out.println("\nCara Stream API (Cari 'Erick'):");
        Mahasiswa targetStream = list.stream()
                .filter(m -> m.getNama().equalsIgnoreCase("Erick"))
                .findFirst()
                .orElse(null);
        System.out.println(targetStream != null ? targetStream : "Data
tidak ditemukan");
    }
```

```
}
```

Output :



C. Unguided

1. Pengelolaan Catatan Hasil Eksperimen Model Machine Learning

Seorang data scientist sedang membandingkan beberapa model machine learning untuk memprediksi data penjualan. Setiap model yang selesai dilatih akan dicatat hasilnya. Agar mudah dianalisis, ia meminta dibuatkan program Java untuk mengelola catatan hasil eksperimen tersebut.

Sistem ini mencatat atribut nama model, akurasi, dan waktu pelatihan dalam satuan detik. Kelas model memiliki static variable untuk menghitung jumlah eksperimen yang dibuat secara otomatis. Pengurutan akurasi menggunakan Comparable, sedangkan

pengurutan waktu pelatihan tercepat menggunakan Comparator. Program menguji penyaringan akurasi minimal 0.85 serta pencarian model KNN dengan Stream API.

File: HasilEksperimen.java

```
package unguided;

public class HasilEksperimen implements Comparable<HasilEksperimen> {
    private static int jumlahEksperimen = 0;

    private String namaModel;
    private double akurasi;
    private int waktuTraining;

    public HasilEksperimen(String namaModel, double akurasi, int
waktuTraining) {
        this.namaModel = namaModel;
        this.akurasi = akurasi;
        this.waktuTraining = waktuTraining;
        jumlahEksperimen++;
    }

    public static int getJumlahEksperimen() {
        return jumlahEksperimen;
    }

    public String getNamaModel() {
        return namaModel;
    }

    public double getAkurasi() {
        return akurasi;
    }

    public int getWaktuTraining() {
        return waktuTraining;
    }

    @Override
    public int compareTo(HasilEksperimen other) {
        return Double.compare(other.akurasi, this.akurasi);
    }

    @Override
    public String toString() {
        return "Model=" + namaModel + ", akurasi=" + akurasi + ", waktu="
+ waktuTraining + " detik";
    }
```

```
}
```

File: WaktuTrainingComparator.java

```
package unguided;

import java.util.Comparator;

public class WaktuTrainingComparator implements
Comparator<HasilEksperimen> {
    @Override
    public int compare(HasilEksperimen h1, HasilEksperimen h2) {
        return Integer.compare(h1.getWaktuTraining(),
h2.getWaktuTraining());
    }
}
```

File: Main.java

```
package unguided;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<HasilEksperimen> daftar = new ArrayList<>();
        daftar.add(new HasilEksperimen("Logistic Regression", 0.82, 12));
        daftar.add(new HasilEksperimen("Random Forest", 0.91, 45));
        daftar.add(new HasilEksperimen("SVM", 0.87, 60));
        daftar.add(new HasilEksperimen("KNN", 0.79, 5));
        daftar.add(new HasilEksperimen("Decision Tree", 0.84, 8));

        System.out.println("Jumlah eksperimen: " +
HasilEksperimen.getJumlahEksperimen());

        Collections.sort(daftar);
        System.out.println("\nSorted by akurasi");
        daftar.forEach(System.out::println);

        Collections.sort(daftar, new WaktuTrainingComparator());
        System.out.println("\nSorted by waktu training");
        daftar.forEach(System.out::println);

        List<HasilEksperimen> akurasiTinggi = daftar.stream()
                .filter(h -> h.getAkurasi() >= 0.85)
```

```
                .collect(Collectors.toList());
        System.out.println("\nModel dengan akurasi minimal 0.85");
        akurasiTinggi.forEach(System.out::println);

        HasilEksperimen hasilCari = daftar.stream()
                .filter(h -> h.getNamaModel().equals("KNN"))
                .findFirst()
                .orElse(null);
        System.out.println("\nPencarian model KNN");
        System.out.println(hasilCari);
    }
}
```

Output :

```
jurymaxwell@192 modul-09-static-class-dan-collection %  /usr/bin/env /Users/jurymaxwell/.vscode/extensions/redhat.java-1.58.0-darwin-arm64/jre/25.0.4.1-macosx-aarch64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/jurymaxwell/College/Pemrograman\ Berbasis\ Objek/Praktikum/modul-09-static-class-dan-collection/project/target/classes unguided.Main
Jumlah eksperimen: 5

Sorted by akurasi
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=KNN, akurasi=0.79, waktu=5 detik

Sorted by waktu training
Model=KNN, akurasi=0.79, waktu=5 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik

Model dengan akurasi minimal 0.85
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik

Pencarian model KNN
Model=KNN, akurasi=0.79, waktu=5 detik
```

Penjelasan :

Program ini dirancang untuk mencatat dan menganalisis hasil eksperimen model machine learning. Kelas HasilEksperimen merepresentasikan entitas model dengan atribut nama, akurasi, dan waktu pelatihan. Atribut static jumlahEksperimen digunakan untuk menghitung total objek eksperimen yang dibuat. Pengurutan alami berbasis akurasi tertinggi diterapkan melalui implementasi interface Comparable pada kelas entitas. Sementara itu, kelas WaktuTrainingComparator mengimplementasikan interface Comparator untuk mengurutkan durasi tercepat.

Pada kelas Main, lima objek eksperimen dimasukkan ke dalam List berpenyandi generic. Program memanggil static method getJumlahEksperimen dan mencetak total sebanyak 5 eksperimen. Koleksi kemudian diurutkan berdasarkan akurasi tertinggi dengan Random Forest 0.91 pada posisi pertama. Pengurutan kedua menggunakan comparator menghasilkan model KNN 5 detik sebagai model tercepat. Operasi stream memfilter dua model dengan akurasi minimal 0.85 serta menemukan model KNN.

D. Kesimpulan

Praktikum Modul 9 memberikan pemahaman mendalam tentang penggunaan static variable dan static method. Komponen static sangat efektif untuk melacak status global atau menghitung instansiasi objek. Selain itu, Java Collection Framework mempermudah pengelolaan sekumpulan objek secara dinamis.

Penerapan interface Comparable dan Comparator memungkinkan pengurutan data yang terstruktur dan fleksibel. Comparable mendefinisikan urutan alami default, sedangkan Comparator menyediakan pengurutan alternatif. Dukungan Stream API dan ekspresi lambda mempermudah operasi penyaringan serta pencarian data koleksi. Kombinasi fitur ini menghasilkan kode Java yang lebih bersih dan efisien.